Valora — Design Document

AI-Powered Real Estate Property Valuation Assistant · Ames Housing dataset · architecture diagram: `architecture/Architecture_Diagram.pdf`

This document explains *why* the system is shaped the way it is: the decisions taken, the alternatives considered and the trade-offs accepted.

1. Problem statement

Real-estate professionals repeatedly ask three questions:

1. *A property has this location, size, age and room count. What similar properties have sold?*
2. *Two similar properties sold for very different prices. What explains the gap?*
3. *What is this property worth, and how sure are we?*

Answering them by searching historical records by hand is slow. The answer is also hard to defend: a number without its comparables, adjustments and caveats is not useful to a valuer. All three questions need the same evidence, the most comparable historical sales, and differ only in what is computed on top. Valora is therefore built as **one retrieval core with specialised agents on top**, not three separate features.

Design goals

- Natural-language input with optional hard filters (neighborhood, year, bedrooms, size, quality, price).
- Every number in an answer traceable to a record or a computation: **no hallucinated figures**.
- Explainable: comparables with reasons and $ adjustments, model feature contributions, risk drivers, judge verdict.
- Robust: works without an LLM key, survives provider outages, blocks unsafe input.
- Measurable: an evaluation suite with hold-out ground truth that can gate changes.

2. Architecture design

The system has three lanes (see the diagram):

Offline ingestion (startup, cached by data hash). Ames ARFF → preprocessing (typing, code mapping, outlier removal, neighborhood statistics) → **chunking** into one natural-language "property case" per sale → **embeddings** (bge-small-en-v1.5) → **FAISS vector DB**, plus a BM25 keyword index over the same chunks. In parallel, a numeric feature frame trains the gradient-boosted **pricing model** and a **hedonic OLS** model, and feeds the metadata store used for hard filters.

Online request. FastAPI receives the query and runs a LangGraph `StateGraph` of eight agents: Guardrail → Classifier → Retriever → Comp Analyst → Pricing Model → Valuer → Risk Analyst → Judge. The judge can send the request back to the retriever once. The response envelope carries every intermediate artifact and a timed trace. The Next.js UI renders it.

Quality loop. Each response is logged with its request id. User feedback (👍 / 👎, corrected value) joins to that id, and corrections become evaluation cases. The built-in evaluator and DeepEval write results that `/api/v1/evaluation/latest` serves to the Evaluation dashboard.

Decision	Rationale	Trade-off
Microservice backend (FastAPI) + separate UI	Clean API contract (11 endpoints, OpenAPI), testable without the UI, any client can use it. Pydantic schemas mirror `frontend/src/lib/types.ts` field for field.	Two processes to run locally (documented, two commands).
All indexes and models in memory, built at startup	2,927 records: embeddings are 4.5 MB, BM25 is about 1 s to build, the model is cached. Sub-200 ms responses in rules mode.	Each worker holds its own copy; fine for one container, revisit beyond ~10^5 records.
Deterministic core, LLM at the edges	Numbers come from code (retrieval, adjustments, model, ensemble); the LLM classifies, phrases and judges.	Rules-mode prose is plainer.
SQLite for requests and feedback	Zero-ops, file-based, enough for an evaluation build; the store class is the swap point for Postgres.	Single writer.

3. Technology selection

3.1 LLM: Claude (Anthropic), Haiku 4.5 by default

The LLM does three narrow jobs per query: structured classification (intent, property type, attributes), narrative phrasing from computed numbers, and judging. In total that is about 3,350 input and 450 output tokens. That shape favours a **fast, cheap model that is reliable at structured JSON**.

Model (gateway id)	₹ / query	Queries per ₹50	Assessment
claude-haiku-4.5	**0.54**	**~92**	**Default for all steps.** Fast, strong at extraction and JSON, native Messages API
claude-sonnet-4.6	1.63	~30	Fallback model; better judge notes; optional for the judge step on panel day
claude-opus-4.8	2.71	~18	Highest quality, slowest; unnecessary for these short tasks
claude-haiku-5.5 / sonnet-5.5	0.05 / 1.08	~920 / ~46	Only on the gateway's chat format; supported via `LLM_API_FORMAT=chat`

Model ids per step come from environment variables (`LLM_MODEL`, `LLM_FAST_MODEL`, `LLM_FALLBACK_MODEL`), so the choice changes without a code change. Both the Anthropic Messages format and OpenAI-style chat completions are supported, because gateways expose different models on each.

3.2 Embedding model: BAAI/bge-small-en-v1.5 (fastembed, ONNX, 384-d)

Chosen **by measurement**. Four encoders were benchmarked in the production index on 55 golden queries (details in the Evaluation Report):

Encoder	Dense P@10	Dense MRR	Hybrid P@10	Query latency
LSA (TF-IDF → SVD), fallback	0.566	0.723	0.647	5 ms
all-MiniLM-L6-v2	0.534	0.695	0.636	7 ms
bge-small-en-v1.5	**0.649**	**0.753**	**0.658**	10 ms
bge-base-en-v1.5 (768-d)	0.627	0.742	0.634	27 ms

bge-small wins on every ranking metric (precision@10, MRR, hit@3). It is 2.5× faster than bge-base and about a third of the memory, and fastembed runs it on CPU without PyTorch (~130 MB RAM). If the model cannot be downloaded, the index falls back to LSA so the service still starts. `/health` reports which encoder is active.

3.3 Vector database: FAISS (`IndexFlatIP`)

Option	Verdict
FAISS `IndexFlatIP` (chosen)	Exact cosine search (vectors are L2-normalised), in-process, no server. Metadata filters are pushed **into** the search with an `IDSelectorBatch`, so top-k is computed only over records that satisfy the hard constraints. Sub-millisecond at 2.9k vectors. `IndexHNSWFlat` is a one-line swap if the corpus grows by orders of magnitude.
ChromaDB	Good developer experience, but adds a persistence layer and its own filter language for no retrieval gain at this scale.
pgvector / Qdrant	Right for multi-service production at millions of rows, but adds a network hop and operational burden here.

The switch from a plain numpy matrix to FAISS was verified to give **identical** evaluation results. If faiss cannot be imported, the store falls back to numpy with the same results.

3.4 Frameworks

Concern	Choice	Why
API	FastAPI + Pydantic v2 + Uvicorn	typed contracts, automatic OpenAPI/Swagger, async with thread-pool offload
Keyword search	rank-bm25	small, pure Python, standard Okapi BM25
ML	scikit-learn `HistGradientBoostingRegressor`, numpy/scipy OLS	strong tabular baseline, no GPU, fast training (≈2 s)
LLM SDK	`anthropic` with a provider-agnostic wrapper	structured outputs, retries, fallback, circuit breaker
Orchestration	LangGraph `StateGraph` over a typed dataclass state (`graph.py`)	conditional edges and a bounded cycle (judge → retriever) as first-class concepts; agents stay plain functions; see §5.5
Evaluation	DeepEval + built-in runner, pytest	LLM-graded and key-free layers

Concern	Choice	Why
Front end	Next.js 16, TypeScript, Tailwind (approved by faculty in place of Streamlit)	evidence-rich UI: drill-downs, compare view, evaluation dashboard, dark mode, mobile

4. Retrieval strategy

4.1 Chunking

One record = one chunk. A sale is an atomic unit of evidence; splitting it would separate a price from the attributes that explain it, and a chunk of 60–80 tokens is well within any encoder's window. Each record is rendered as a natural-language **property case** (`loader.describe`):

> 3-bedroom, 1-bath single-family home in North Ames (NAmes) with 1,656 sq ft of living area above grade, a 1,080 sq ft basement and a 2-car garage of 528 sq ft. Built in 1960; 50 years old at sale (older). Overall quality 6/10, condition 5/10, 7 rooms total, on a 31,770 sq ft lot. Sold in 2010 for $215,000.

Dense encoders are trained on prose, not column vectors. The text deliberately includes **both** the neighborhood code and its full name, numbers in plain form, and derived words (age at sale, an era word, remodel year), because those are what users type. The numeric frame is kept alongside the text for filtering, adjustments and the model.

4.2 Embedding approach

Each property case is embedded once with bge-small-en-v1.5 (L2-normalised, 384-d) and cached on disk under a hash of the model name and corpus text. Queries are embedded with the same model at request time (~10 ms).

4.3 Retrieval method and hybrid search

1. **Metadata filtering first (hard constraints).** Neighborhood, year-built range, bedroom range, living-area range, minimum quality and maximum price restrict the candidate set *before* ranking. A user who says "built before 1940" must never see a 2005 home. The response reports the surviving pool size, so thin results are explainable.
2. **Dense search** in FAISS over the allowed ids → top 40.
3. **Keyword search** with BM25 over the same allowed ids → top 40. Tokenisation expands neighborhood names to codes and back.
4. **Reciprocal-rank fusion** (k = 60, dense weight 0.65). RRF needs no calibration between cosine and BM25 scales and is robust if either ranking is noisy. Per-hit semantic, keyword and hybrid scores are kept for transparency.
5. **Comparable prioritization** (Comp Analyst). Text similarity is not appraisal comparability, so the 40 candidates are re-scored on appraiser criteria and the top-k kept (default 8):

Criterion	Weight
Same neighborhood	0.30
Living area within 10 % (within 20 %)	0.22 (0.12)
Built within 8 years (within 20)	0.15 (0.07)
Identical quality grade (±1)	0.12 (0.07)
Same bedroom count	0.06
Recent sale (last two years of data)	0.04
Same garage capacity	0.03

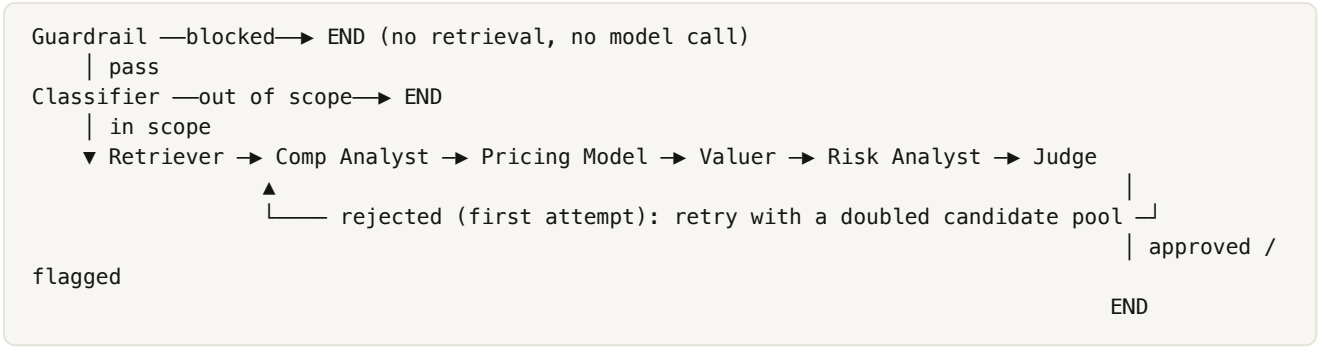
Priority = 0.85 × appraisal score + 0.15 × hybrid retrieval score. Each comparable then gets **appraisal-style $ adjustments** toward the subject (living area at 55 % of its $/sq ft, quality, age, garage, basement, using rates from the hedonic fit), producing an adjusted price. Every reason and adjustment is returned to the UI.

5. Agent design

5.1 Roles

Agent	Role	LLM?	Fallback
Guardrail	NFKC normalisation, control-char stripping, length bounds, prompt-injection scan, PII redaction (email, SSN, phone) before any model sees the text, domain relevance; later clamps extracted attributes to the dataset's range	No	—
Classifier (property classification agent)	intent (valuation, similar_properties, comparison, price_difference, market_insight, out_of_scope), property type (single-family, townhouse, duplex), attributes	Yes, structured output	regex extraction; regex always back-fills missing attributes; explicit filters override
Retriever	hybrid search with hard filters (§4.3)	No	LSA encoder if fastembed is unavailable
Comp Analyst	appraisal prioritization + $ adjustments	No	—
Pricing Model	ML prediction, 90 % interval, feature contributions	No	—
Valuer	ensemble of comparable-weighted estimate and ML prediction; range, confidence, $/sq ft; narrative	Narrative only	template narrative from the same numbers
Risk Analyst	value-risk score 0–100 from comparable dispersion, neighborhood coverage, method divergence, neighborhood volatility, age, quality	No	—
Judge (LLM-as-judge)	validates the valuation on five criteria; verdict approved / flagged / rejected	Yes, independent prompt	deterministic checks alone

5.2 Workflow



The **judge → retriever retry** makes the collaboration real rather than a linear pipeline. The most common cause of rejection is thin comparable coverage, and widening the pool is the right remedy. The retry is bounded to one attempt to cap latency.

5.3 Agent communication

Agents communicate through a **typed shared state** (`graph.State`): each node reads the fields it needs and writes its outputs (Pydantic models such as `Classification`, `Comparable`, `Valuation`, `RiskAssessment`, `JudgeVerdict`). There is no free-text hand-off between agents; LLM outputs are parsed into schemas before they enter the state. Every node execution is appended to a trace with duration, tool and an output summary, returned to the client as `agentTrace`.

5.4 The judge

The judge sees only the evidence (subject, comparables, valuation, model prediction, answer). Its five criteria each have a deterministic check:

Criterion	Deterministic check	Weight
Comparable relevance	≥ 3 comparables, ≥ 2 in the subject's neighborhood	0.30
Model agreement	ensemble vs ML estimate within 10 %	0.20
Adjustment reasonableness	net adjustment ≤ ±30 % of sale price on every comparable	0.15
Grounded narrative	every figure ≥ 1,000 in the answer matches evidence within 1 %	0.20
Range calibration	interval width 5–35 % of the estimate	0.15

In LLM mode the final score is the average of the LLM's score and the deterministic score, and an ungrounded figure always forces **rejected**. The LLM can add judgement but cannot approve what the checks fail. Verdict: approved ≥ 0.8, flagged ≥ 0.5, otherwise rejected.

5.5 Orchestration with LangGraph

The graph is a LangGraph `StateGraph` compiled once at start-up: eight nodes, three conditional edges (Guardrail → END, Classifier → END, Judge → Retriever) and a recursion limit as a hard stop on the loop. LangGraph gives us the cycle and the routing as declared structure rather than control flow, and the compiled graph can be inspected and drawn (`GRAPH.get_graph()`), which is what the topology test checks.

We deliberately keep the framework at the edges:

- **Agents are plain functions** `State -> State` with no LangGraph imports, so each is unit-testable on its own and the business logic does not depend on the orchestrator.
- **One wrapper** (`_traced`) turns an agent into a node: it times the call, appends an `AgentStep` to the trace and records a failure as an `error` step instead of aborting the request.
- **Routers are pure.** Decisions that change state (e.g. the retry counter) are made inside the Judge node and stored on the state (`retry`, `attempt`); the routing functions only read them.
- **No checkpointer.** Each request is independent and finishes in seconds, so persistence would add cost without benefit. Adding one later (for human-in-the-loop review of flagged valuations) is a one-line change.

We considered CrewAI and AutoGen. Both are built around conversational, LLM-driven hand-offs; here five of the eight agents are deterministic (retrieval, ML, statistics) and the flow is known in advance, so an explicit graph with typed state is the better fit and keeps the system reproducible in rules mode.

6. Guardrails, security and privacy

- **Input:** normalisation, length limits (600 characters processed; 2,000 maximum accepted), prompt-injection patterns, PII redaction **before** any LLM call, domain-relevance gate, the classifier's `out_of_scope` intent as a semantic second gate, and attribute clamping to the dataset's observed range (reported as a warning).
- **Output:** Pydantic validation of every LLM response, the judge's grounding check, and numbers never produced by the LLM.
- **API:** validation on every request (422), RFC 9457 problem-details errors, per-IP rate limiting (60/min), explicit CORS origins.
- **Secrets:** only from environment variables / `.env` (git-ignored); `.env.example` documents them.

7. Evaluation strategy

Layer	Question	Metrics
Built-in runner (no key, CI-gating)	Does it work?	valuation MAPE, range coverage, contextual precision/recall, faithfulness (groundedness), hallucination, answer relevancy, intent and guardrail accuracy, judge agreement, latency
Embedding benchmark	Which encoder?	P@10, MRR, hit@3, comparable-price error, latency
Model CV	Is the predictor sound?	5-fold MAPE, R^2
DeepEval (LLM-graded)	Is the recommendation good?	Faithfulness, Answer Relevancy, Contextual Precision/Recall, Hallucination, custom ValuationMAPE
Online judge + feedback	Is this answer good?	verdict per response; human corrections become new test cases
pytest	Do components and failure paths behave?	64 tests

The golden set has 60 cases: 44 hold-out valuations generated from real records, with each subject excluded from its own comparables and from the pricing model's training data, plus 11 intent/retrieval and 5 guardrail cases. Current results: **MAPE 9.6 %, 80 % range coverage, contextual precision 91 % / recall 97 %, 0 % hallucination, intent 98 %, guardrails 100 %** (full detail in `docs/Evaluation_Report.pdf`).

8. Challenges and trade-offs

Challenge	What we did	Trade-off / limitation
Tabular data does not embed well	Rendered each row as a natural-language property case including code + name and derived words	The template is a tuning surface; changes need re-embedding (automatic via cache key)
Text similarity ≠ appraisal comparability	Separate retrieval score from comparable priority; show both	Two scores to explain
LLMs invent numbers	LLM never computes; grounding check; judge rejects ungrounded figures	Rules-mode prose is plainer
Gateway stripped structured outputs	Self-healing JSON-by-instruction fallback with schema validation and one repair call	~40 extra prompt tokens in fallback mode
₹50 LLM budget	Haiku 4.5 default; full rules mode; circuit breaker (402/403 or 4 failures → 60 s rules)	Rules-mode intent handling is weaker on unusual phrasing
Thin neighborhoods (1–10 sales)	Fallback to similar homes elsewhere; risk score and confidence reflect it	Lower confidence, by design
Linear adjustments	Rates from the hedonic fit; judge flags > ±30 % net	Interactions are left to the boosted model
No time adjustment	Recency bonus only	Valuations are in 2006–2010 dollars
Explainability without SHAP	Single-feature substitution against the median ("SHAP-lite")	Approximate attributions

Next steps: stream the agent trace over Server-Sent Events; a "confidence too low → ask a clarifying question" branch; time-adjusted comparables; SHAP once the dependency footprint is acceptable; nightly DeepEval with regression alerts; judge calibration against human corrections.